Markdown
```
# TECHNICAL SPECIFICATION BRIEF: GoOuts V8 Hybrid Ledger & Automated Infrastructure
**Document ID:** GoOuts_Sponsor_Brief_V8_Hybrid
**Classification:** Confidential - Internal Engineering & Architecture Review Only
**Target Architecture:** Firebase Ecosystem / Stripe Issuing API / Meta Graph API / Open Banking v3.3
**Date:** June 2026

---

## SECTION 1: SYSTEM ARCHITECTURE & EXCLUSIONS

### 1.1 The Regulatory Perimeter (TSP Exclusion)
GoOuts does not sit within the regulated flow of funds under FCA guidelines. The platform operates explicitly under the **Technical Service Provider (TSP)** exclusion.
* All core transactional funding, card issuing, and primary ledger balances are structurally managed by Stripe (an authorized Electronic Money Institution).
* All merchant account collections are initiated as payment instructions via Open Banking APIs rather than GoOuts holding or pooling client money.

### 1.2 Apple Wallet Integration Dynamics
Platform unit economics are completely protected against transaction network dilution inside the UK/European market.
* Under UK/EU interchange fee regulations, card-issuing interchange margins are strictly capped at a maximum of 0.20% for consumer debit cards.
* Consequently, Apple charges GoOuts a **0% transaction fee** for tokens provisioned inside the Apple Wallet.
* **Tokenization Overhead:** Stripe manages the cryptographic token framework directly with Visa/Mastercard networks. GoOuts incurs a microscopic, single-instance network token pass-through fee (approx. 1p to 2p) strictly at the moment the consumer clicks *"Add to Apple Wallet"*. All subsequent retail POS taps are 100% free of Apple network fees.

---

## SECTION 2: THE 4-PILLAR HYBRID BALANCE LEDGER

To bypass traditional disk-bound database performance constraints during a live card transaction, user balances are split across 4 logic pillars mirrored inside a real-time, low-latency framework using Firebase Realtime Database (RTDB) and Cloud Firestore.

                [ 4-PILLAR MATRIX ARCHITECTURE ]

┌──────────────────────────┬──────────────────────────┐
│ PILLAR 1: CASHBACK WALLET │ PILLAR 2: DEPOSITED WALLET │
│ • Read-optimized from RTDB │ • User funded via Open Banking │
│ • Derived from merchant promos │ • Primary local cash layer │
├──────────────────────────┼──────────────────────────┤
│ PILLAR 3: WELCOME BONUS │ PILLAR 4: THE OPEN BANKING GAP │
│ • Platform-funded incentive │ • Dynamic fallback calculation │
│ • Hard expiration logic attached │ • Variable Recurring Payment │
└──────────────────────────┴──────────────────────────┘
```

2.1 Live Balancing Formula
When a Stripe real-time authorization request arrives, the backend calculates the allocation sequence in strict priority order using the following mathematical logic:

$$TotalTransaction = P_1 + P_2 + P_3 + P_4$$

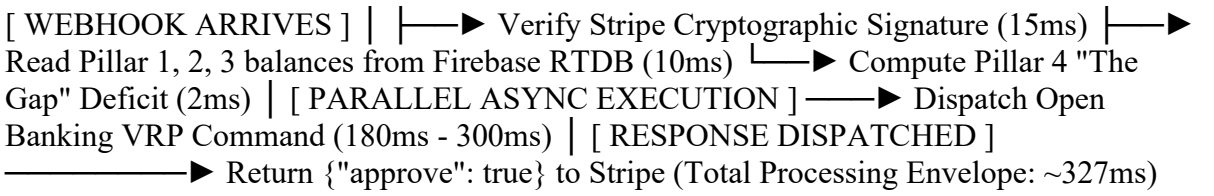
Where:
* P_1 = Available Cashback Balance (Deducted first)
* P_2 = Available Deposited Wallet Balance (Deducted second)
* P_3 = Active Welcome Bonus Balance (Deducted third)
* P_4 = The remaining outstanding deficit dynamically pulled via a Variable Recurring Payment (VRP) instruction from the user's connected high-street bank account.

SECTION 3: STRIPE 2-SECOND REAL-TIME AUTHORIZATION TIMEOUT CONTROL

3.1 The Low-Latency Timeout Challenge
Stripe Issuing real-time authorization webhooks are strictly synchronous. When a user taps their phone at a point of sale, **Stripe pauses the authorization lifecycle and allows the GoOuts server exactly 2.00 seconds to respond** with `{"approve": true}` or `{"decline": true}`. Responses exceeding 2.00 seconds cause a network timeout and default transaction failure.

3.2 Engineering Protocols for Sub-Second Processing
To ensure execution times remain under **400ms**, developers must implement the following concurrent architecture:

1. **No Cold Queries:** Do not read or write to standard relational disks during the live window. Mirror active balances in **Firebase Realtime Database (RTDB)** for in-memory lookups (<15ms).
2. **Concurrent Execution:** Execute all internal pillar verifications and signature checks asynchronously using concurrent patterns (e.g., `Promise.all` or `async/await` parallelism).
3. **Optimistic Open Banking Auth:** When calculating P_4 (The Gap), the system fires the VRP call to the Open Banking provider (TrueLayer/Yapily). The system **must not wait** for the actual bank settlement to complete. As long as the bank API responds with a successful *initiation command acknowledgement*, the function must instantly return `{"approve": true}` to Stripe. Your platform corporate operational float inside Stripe absorbs the millisecond buffer.



SECTION 4: POST-SWIPE DEFERRAL WORKER QUEUES

To guarantee timeout windows are never breached, developers must segregate backend logic into **Synchronous (Inline)** and **Asynchronous (Deferred)** actions.

Execution Layer	Action Type	Tasks Assigned
Synchronous	Inline (<400ms)	Signature validation; RTDB balance check; VRP Gap initialization; Stripe response dispatch.
Asynchronous	Background Worker Queue	B2B Open Banking 20% Merchant Clawback; P1 Cashback Generation Loop; Permanent Firestore Audit Writing; Social Engine Activation.

SECTION 5: THE SOCIAL BOOST CAMPAIGN ENGINE (AUTOMATED VERIFICATION)

5.1 Architectural Overview
The Social Boost layer turns users into micro-influencers at the point of sale. When activated, users earn an extra 10% (or merchant-defined) **Verified Cashback** by sharing their transaction natively on Instagram or Facebook.

To eliminate conversion friction, this engine is **fully automated and serverless**. It bypasses the need for users to manually upload screenshots or video files, relying on real-time asynchronous webhooks delivered directly by the Meta Graph API.

5.2 Data Model Extensions (Cloud Firestore)

`/users/{userId}`
```json
{
  "instagramHandle": "@john_doe", // Captured once on initial feature opt-in
  "facebookHandle": null
}
```

/transactions/{transactionId}

JSON
```
{
  "transactionId": "TXN_STRIPE_99281",
  "userId": "USER_4112",
  "merchantId": "MERCH_7731",
  "amounts": {
    "grossSale": 10.00,
    "baseCashback": 1.50,
    "socialBonus": 1.00
  },
  "socialCampaign": {
    "isOptedIn": true,
    "userHandle": "@john_doe",
    "requiredTags": ["@GoOuts_App", "@MerchantCafeHandle"],
    "verificationStatus": "PENDING_VERIFICATION", // PENDING_VERIFICATION | VERIFIED_AND_RELEASED | EXPIRED
    "metaMediaId": null
  }
}
```

5.3 Front-End Implementation & Native Share Sheets

1. **Handle Enforcement:** If `user.instagramHandle == null`, the frontend client must present a text-input modal capturing the account handle before routing to the share step.
2. **Clipboard Payload Automation:** Upon clicking "Share & Claim", the application must programmatically overwrite the device clipboard with a standardized text array via the OS clipboard API:
    - *Template Asset:* `"Loving my morning coffee at @MerchantCafeHandle! Powered by @GoOuts_App ✨🚀"`
3. **OS Intent Chooser:** Invoke the native **iOS `UIActivityViewController`** or **Android `Intent.createChooser`** to launch the destination social network app directly into the user's Story/Feed composer screen. Display a temporary UI toast: *"Caption auto-copied! Hold down and tap paste to include required tags."*

5.4 Back-End Automation Loop & Meta Integration

1. The Serverless HTTP Endpoint

Expose a single public **Firebase Cloud Function (HTTPS Trigger)** to serve as the Meta Webhook listener destination. Subscribe this endpoint to the `mentions` edge within the Facebook Developer Console.

2. The Verification Execution Script (Node.js / Firebase Functions)

JavaScript

```
exports.metaSocialWebhook = functions.https.onRequest(async (req, res) => {
  // 1. Validate the Meta X-Hub-Signature header for security
  if (!validateMetaSignature(req)) return
res.status(401).send('Unauthorized');

  const { object, entry } = req.body;
  if (object !== 'instagram') return res.status(200).send('OK');

  // 2. Extract payload variables from Meta Graph API JSON structure
  const incomingHandle = "@" + entry[0].changes[0].value.username;
  const mediaId = entry[0].changes[0].value.media_id;

  try {
    // 3. Query Firestore for an active transaction matching the handle
within a 60-minute window
    const oneHourAgo = new Date(Date.now() - 60 * 60 * 1000);
    const txnSnapshot = await db.collection('transactions')
      .where('socialCampaign.userHandle', '==', incomingHandle)
      .where('socialCampaign.verificationStatus', '==',
'PENDING_VERIFICATION')
      .where('timestamp', '>=', oneHourAgo)
      .limit(1)
      .get();

    if (txnSnapshot.empty) {
      console.log(`No matching pending transaction found for handle:
${incomingHandle}`);
      return res.status(200).send('OK');
    }

    const txnDoc = txnSnapshot.docs[0];
```

```
    const txnData = txnDoc.data();

    // 4. Atomic Ledger Update Phase
    const batch = db.batch();

    // Flip status to verified
    batch.update(txnDoc.ref, {
      'socialCampaign.verificationStatus': 'VERIFIED_AND_RELEASED',
      'socialCampaign.metaMediaId': mediaId
    });

    // Release the 10% frozen allocation out of Merchant Social Float into
User Spendable Balance
    const userWalletRef = db.collection('wallets').doc(txnData.userId);
    batch.update(userWalletRef, {
      'spendableBalance':
admin.firestore.FieldValue.increment(txnData.amounts.socialBonus)
    });

    await batch.commit();

    // Note: Due to Firestore's real-time listeners (onSnapshot), the
user's mobile interface
    // will instantly render the success state/confetti animation with zero
pull-to-refresh overhead.

    return res.status(200).send('OK');
  } catch (error) {
    console.error('Execution failure in Social Matching Engine:', error);
    return res.status(500).send('Internal Server Error');
  }
});
```

SECTION 6: THREE-WAY STATEMENT LOG RECONCILIATION

To ensure accurate auditing across the platform, transactions using the Social Boost framework must generate matching, transparent data sets across all three platform interfaces.

6.1 Consumer App UI History Ledger

```
The Daily Grind Café
Today at 14:32 — Apple Wallet Authorization
-----------------------------------------------------
Total Purchase Amount:                        £10.00
Base Cashback Earned (15%):                   +£1.50
Verified Social Boost Bonus (10%):            +£1.00  [🟢 AI Verified via
Meta]
-----------------------------------------------------
Net Balance Deposited to GoOuts Wallet:       +£2.50
```

6.2 Merchant Portal Statement Log

```
Transaction Reference: #TXN_STRIPE_99281
Type: Customer Visit & Social Marketing Match
-----------------------------------------------------
Gross Retail Sale via Card Network:           +£10.00
```

```
Base Platform Discount Fee (20% Outflow):     -£2.00
Social Boost Match Outflow (10% Bonus):       -£1.00
Social Boost Tech Processing Fee:             -£0.10
-----------------------------------------------------
Net Campaign Balance Impact:                  -£3.10
[Result: 1 Verified Paying Customer Generated + 1 Live Instagram Brand
Asset Placed]
```

6.3 GoOuts Master Financial Admin Matrix

```
[ MASTER AUDIT RECORD: #TXN_STRIPE_99281 ]

FUND INFLOWS:
  * Open Banking B2B Base Discount Fee Pull:   +£2.00
  * Open Banking B2B Social Match Float Pull:  +£1.10
  * Card Interchange Revenue Allocation:       +£0.02
  ---------------------------------------------------
  TOTAL INFLOW VALUE:                          +£3.12

FUND OUTFLOWS:
  * Consumer Base Cashback Allocation (P1):    -£1.50
  * Consumer Social Reward Allocation:         -£1.00
  * Open Banking Network Routing/Compute Cost: -£0.02
  ---------------------------------------------------
  TOTAL OUTFLOW VALUE:                         -£2.52

=====================================================
🟢 NET RETAINED PLATFORM MARGIN (PROFIT):     +£0.60
=====================================================
```